

# SLI / SLO 설계 문서

이 문서는 강의 카탈로그 API의 정상 상태를 정의하고, 그 기준을 Prometheus/Grafana로 측정하는 방법을 정리합니다.  
핵심 원칙:

서비스 정상 여부는 프로세스 생존이 아니라 사용자 API의 성공률, 오류율, 지연 시간으로 판단한다.

## 1. 서비스와 범위

| 항목      | 내용                                             |
|---------|------------------------------------------------|
| 가상 서비스  | Elice 강의 카탈로그 조회 API                           |
| 사용자 API | GET /api/v1/courses , GET /api/v1/courses/{id} |
| 트래픽 특성  | 동기 read-heavy 조회                               |
| 의존성     | Postgres 1개                                    |
| 실행 환경   | Docker Compose 단일 인스턴스                         |

API 서버는 신뢰성 검증 대상입니다. 비즈니스 로직은 최소화하고, 정상 응답·지연·5xx·Postgres 장애를 재현할 수 있게 구성했습니다.  
Postgres는 기능 확장이 아니라 의존성 장애 전파를 보여주기 위한 최소 구성입니다. ORM, migration tool, cache, HA는 범위에서 제외했습니다.

## 2. 정상 상태 정의

정상 상태:

사용자가 강의 목록 또는 상세 API를 호출했을 때, 정해진 지연 시간 안에 5xx 없이 응답을 받는 상태.

| Endpoint          | 목적                | SLI 포함 |
|-------------------|-------------------|--------|
| /healthz          | 프로세스 생존 확인        | 제외     |
| /readyz           | DB 연결 가능 여부 확인    | 제외     |
| /metrics          | Prometheus scrape | 제외     |
| /admin/fault-mode | 장애 주입 제어          | 제외     |
| /api/v1/*         | 사용자 요청            | 포함     |

SLI는 항상 사용자 API만 집계합니다.

```
{route=~"/api/v1/.*"}
```

`/healthz` 가 200이어도 사용자는 5xx를 받을 수 있습니다. 예를 들어 Postgres가 중지되면 `/healthz` 는 성공하지만 `/readyz` 는 503이고 사용자 API는 5xx로 실패합니다. 따라서 정상성은 probe가 아니라 실제 사용자 API 결과로 판단합니다.

Docker Compose에는 readiness 결과로 트래픽을 차단하는 routing 계층이 없습니다. 이 한계는 incident report의 contributing factor로 다룹니다.

### 3. SLI / SLO

| SLI          | 정의                         | SLO         | 역할           |
|--------------|----------------------------|-------------|--------------|
| Availability | 5xx가 아닌 사용자 응답 / 전체 사용자 응답 | 30일 99.9%   | 장기 신뢰도, 에러버짓 |
| Error Rate   | 5xx 사용자 응답 / 전체 사용자 응답     | 5분 1% 이하    | 단기 장애 감지     |
| Latency p95  | 사용자 API 응답 시간 p95          | 5분 300ms 이하 | 사용자 체감 지연    |

Availability와 Error Rate는 수식상 연결되지만 목적이 다릅니다. Availability는 30일 목표이고, Error Rate는 지금 장애가 발생 중인지 보는 운영 신호입니다.

대표 PromQL:

```
# 30일 Availability
sum(increase(http_requests_total{route=~"/api/v1/.*"},status!~"5..")[30d]))
/
sum(increase(http_requests_total{route=~"/api/v1/.*"}[30d]))

# 5분 Error Rate
sum(rate(http_requests_total{route=~"/api/v1/.*",status=~"5.."}[5m]))
/
sum(rate(http_requests_total{route=~"/api/v1/.*"}[5m]))

# 5분 Latency p95
histogram_quantile(0.95,
  sum by(le) (rate(http_request_duration_seconds_bucket{route=~"/api/v1/.*"}[5m]))
)
```

데모 환경은 장기 보관을 구성하지 않았으므로 dashboard에서는 30일 대신 1시간 proxy를 표시합니다.

SLO 근거:

| 항목                 | 결정 근거                                              |
|--------------------|----------------------------------------------------|
| Availability 99.9% | 월 약 43분 장애 허용. 단일 incident와 짧은 정비를 흡수할 수 있는 현실적 목표 |
| Error Rate 1%      | 짧은 윈도우에서 사용자 실패가 관측되는 수준                           |
| Latency p95 300ms  | 단순 read API에서 관리해야 할 체감 지연 기준                      |

4xx는 서비스 신뢰성 실패로 보지 않습니다. 401/403 급증이나 비정상 404 증가는 별도 운영 신호로 확장할 수 있습니다.

## 4. Metrics

| 목적         | 메트릭                                                      | 비고                 |
|------------|----------------------------------------------------------|--------------------|
| 요청 수 / 오류율 | <code>http_requests_total{method,route,status}</code>    | 사용자 SLI 기준         |
| 지연 시간      | <code>http_request_duration_seconds{method,route}</code> | p95 계산             |
| 장애 주입 상태   | <code>api_fault_mode{mode}</code>                        | Grafana annotation |
| DB 지연      | <code>db_query_duration_seconds{operation}</code>        | RCA 보조             |
| DB 오류      | <code>db_errors_total{operation}</code>                  | RCA 보조             |

course id마다 시계열이 늘어나는 문제를 피하기 위해 `route` 라벨은 실제 URL이 아니라 route template을 사용합니다.

```
/api/v1/courses/1 -> /api/v1/courses/{course_id}
```

Latency bucket은 SLO와 alert 임계 근처에 맞춥니다.

```
(0.05, 0.1, 0.2, 0.3, 0.5, 1.0, 2.0, 5.0)
```

DB 메트릭은 paging 기준이 아닙니다. 사용자 영향은 Error Rate와 Latency로 감지하고, DB 메트릭은 원인 분석에 사용합니다.

## 5. Alert

Alert는 cause가 아니라 user-facing symptom을 기준으로 둡니다.

| Alert                        | 조건                              | for | Severity | 목적        |
|------------------------------|---------------------------------|-----|----------|-----------|
| <code>APIInstanceDown</code> | <code>up{job="api"} == 0</code> | 1m  | critical | scrape 불가 |
| <code>HighErrorRate</code>   | 5xx 비율 > 5% and RPS > 0.1       | 2m  | critical | 사용자 실패    |
| <code>HighLatencyP95</code>  | p95 > 500ms and RPS > 0.1       | 2m  | warning  | 사용자 지연    |

트래픽이 거의 없으면 비율 신호가 의미 없고, 1건의 실패가 100%처럼 보일 수 있기 때문에 Error Rate와 Latency alert에는 traffic guard를 둡니다.

```
and sum(rate(http_requests_total{route=~"/api/v1/.*"}[2m])) > 0.1
```

SLO와 alert 임계값은 분리합니다.

| 항목         | SLO      | Alert    |
|------------|----------|----------|
| Error Rate | 5분 1% 이하 | 2분 5% 초과 |

| 항목          | SLO         | Alert       |
|-------------|-------------|-------------|
| Latency p95 | 5분 300ms 이하 | 2분 500ms 초과 |

SLO는 장기 목표이고, alert는 운영자 개입 신호입니다. 모든 rule은 `prometheus/alert-rules.yml` 에 코드로 관리합니다.

## 6. Dashboard

Dashboard는 다음 순서로 읽도록 구성했습니다.

| 순서 | 패널                  | 질문                       |
|----|---------------------|--------------------------|
| 1  | Scrape Status       | Prometheus가 API를 관측 중인가? |
| 2  | Request Rate        | 트래픽이 들어오고 있는가?           |
| 3  | Error Rate          | 사용자가 5xx를 보고 있는가?        |
| 4  | Availability        | 최근 성공률은 어떤가?             |
| 5  | Latency p95         | 응답이 느려졌는가?               |
| 6  | Status Distribution | 어떤 status가 늘었는가?         |
| 7  | DB Query Latency    | DB가 느린가?                 |
| 8  | DB Errors Rate      | DB 호출이 실패하는가?            |

`Scrape Status` 는 사용자 가용성이 아닙니다. 사용자 영향은 Error Rate, Availability, Latency 패널로 판단합니다.

`api_fault_mode{mode!="normal"} == 1` 은 Grafana annotation으로 표시합니다. DB 패널은 RCA 안정성을 위해 5분 window를 사용합니다.

모든 datasource와 dashboard는 `grafana/provisioning/` 으로 자동 등록됩니다.

## 7. 장애 검증

문항 10이 요구한 API 3 동작(정상/지연/5xx)을 incident 시나리오와 1:1로 매핑합니다.

| 시나리오            | API 동작       | Trigger                                                               | 기대 신호                                                               | Layer       |
|-----------------|--------------|-----------------------------------------------------------------------|---------------------------------------------------------------------|-------------|
| Latency fault   | 의도적 응답 지연    | <code>POST /admin/fault-mode {"mode":"slow","delay_ms":700}</code>    | <code>HighLatencyP95</code><br>만 firing, Error Rate 정상, DB 오류 없음    | application |
| Application 5xx | 의도적 오류 (5xx) | <code>POST /admin/fault-mode {"mode":"flaky","error_rate":0.3}</code> | <code>HighErrorRate</code><br>만 firing, latency 정상, DB 오류 series 없음 | application |

| 시나리오         | API 동작 | Trigger                                                                 | 기대 신호                                                                                                              | Layer      |
|--------------|--------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|------------|
| DB stop (확장) | 해당 없음  | <code>fault-mode=normal</code> 에서 <code>docker stop sre-postgres</code> | <code>/readyz</code> 503, <code>HighErrorRate</code><br>+<br><code>HighLatencyP95</code><br>동반 firing, DB error 증가 | dependency |

DB stop은 의존성 장애가 사용자 SLI로 전파되는 흐름을 검증하기 위한 확장 시나리오입니다. 같은 `HighErrorRate` alert가 application( `flaky` )과 dependency ( `DB stop` )에서 모두 발생할 수 있으며, dashboard의 DB diagnostic 패널이 두 layer를 구분합니다.

복구는 명령 실행이 아니라 지표로 판단합니다.

- alert resolved
- Error Rate 정상화
- Latency p95 300ms 이하
- DB incident는 `/readyz` 200과 DB error 감소 확인

## 8. 한계와 확장 방향

| 한계                                   | 실서비스 확장                               |
|--------------------------------------|---------------------------------------|
| 단일 인스턴스                              | Kubernetes deployment, rolling update |
| Compose readiness routing 부재         | readiness probe, LB health check      |
| Prometheus 장기 보관 없음                  | remote write, Thanos/Mimir            |
| Alertmanager 없음                      | Alertmanager, notification routing    |
| 단일 임계 alert                          | multi-window multi-burn-rate alert    |
| 구조화 로그/trace 없음                      | OpenTelemetry, log/trace backend      |
| <code>/admin/fault-mode</code> 인증 없음 | 내부망 격리, 인증                            |
| DB timeout 1계층만 적용                   | connect/acquire/statement timeout 분리  |

본 프로젝트는 복잡한 인프라보다 정상/장애 기준 정의와 검증에 집중합니다.